

Universidad Nacional de La Matanza
Departamento de Ingeniería e Investigaciones Tecnológicas

Programación Concurrente
Trabajo práctico – Métricas

(2C2026 – 3658 – Grupo 07)

Profesores
Carnuccio, Esteban
Hirschfeldt, Dario
Palomo, Maximo Facundo
Volker, Mariano Leonardo

Integrantes del grupo 07
Farias, Maira Soledad
Hoz, Lucas Nahuel
Loiero, Federico Nicolas
Massa, Valentin
Niedfeld, Kevin Javier
Zapata, Santiago

Índice

Introducción 3

Desarrollo 3

Resultados 4

Métricas 4

Hallazgos 4

Conclusiones 5

Reflexiones sobre ventajas y limitaciones de usar IA en programación concurrente 5

Referencias 5

Introducción

La programación concurrente permite la ejecución de múltiples tareas intercaladamente, con el objetivo de mejorar la capacidad de respuesta y aprovechar mejor los recursos del sistema.

El desarrollo de sistemas concurrentes presenta desafíos relacionados con la sincronización y el acceso a recursos compartidos. Entre los problemas más frecuentes se encuentran las condiciones de carrera, los Deadlocks y la inanición. Para prevenirlos se utilizan mecanismos como hilos de ejecución, Mutex, semáforos y colas sincronizadas.

Por otra parte, la generación automática de código mediante inteligencia artificial permite generar soluciones a partir de instrucciones en lenguaje natural, conocidas como prompts. Sin embargo, el código generado requiere una revisión de un experto para identificar errores funcionales, problemas de sincronización y limitaciones de rendimiento.

Para evaluar la calidad del software generado se utilizan métricas que permiten analizar distintas mediciones. Estas mediciones permiten identificar debilidades en la solución, así como analizar su eficiencia y comportamiento.

En este contexto, este informe busca evaluar la capacidad de las herramientas de inteligencia artificial para generar software concurrente, considerando tanto su funcionamiento como su eficiencia en el uso de los recursos del sistema. Durante la elaboración del informe se experimentó con la generación de código a través de herramientas de VibeCoding, analizando el resultado obtenido.

Desarrollo

Para experimentar con VibeCoding se creó un clon del juego de Pac-Man utilizando la librería Pygame de Python. Se le pidió a la IA una versión secuencial y una versión implementando mecanismos de concurrencia creando Workers de cada fantasma.

A continuación, los enlaces al Zero-Shot Prompt técnico y al código generado por el modelo (Astra 6, esfuerzo medio) de OpenAI:

- Prompt: https://github.com/UNLAM-PROG-C/2026-PROGC-Q2-M7/blob/develop/TP%20Metrica/prompt.md
- Código generado: https://github.com/UNLAM-PROG-C/2026-PROGC-Q2-M7/tree/develop/TP%20Metrica

Aunque inicialmente parece que el código generado por la IA cumple con los requerimientos definidos en el prompt, tanto la UX como las funcionalidades presentan claros errores que la IA no tuvo en cuenta.

Resultados

Métricas

En los ensayos de ejecución secuencial y concurrente del juego:

- La versión concurrente actualiza la UI y responde 24 veces más rápido a las entradas del usuario que la versión secuencial.
- La versión concurrente alcanzó un pico del 28% de la CPU, frente al 37% de la versión secuencial.
- La versión concurrente utilizó todos los hilos del procesador, mientras que la versión secuencial solo utilizó un único hilo.

Estas métricas se realizaron sobre una MacBook Pro (edición 2020) con un procesador Intel Core i7 y 16 GB de RAM. Para más información sobre el nivel de satisfacción de las métricas establecidas para este ensayo, referirse al Excel de métricas (https://github.com/UNLAM-PROG-C/2026-PROGC-Q2-M7/blob/develop/TP%20Metrica/metricas. .xlsx).

Hallazgos

Se han encontrado los siguientes fallos de UX:

- Si el jugador pierde todas las vidas, la leyenda “Enter / Espacio: jugar o continuar” sigue mostrándose.
- Cuando los fantasmas son vulnerables, algunos se fusionan con otros y desaparecen.
- Los fantasmas aparecen muy cerca de Pac-Man, dando poco tiempo de reacción al usuario.

Estos errores no fueron tenidos en cuenta por la IA, ya que corresponden a errores de UX que los modelos no suelen interpretar correctamente al momento de diseñar sistemas que incorporan UIs.

Con respecto al código generado y la estructura del mismo:

- La IA no generó código legible al no agrupar instrucciones relacionadas, sino que las escribió en secuencia, lo que dificulta la lectura en base a competencias.
- No ignoró los resultados de los benchmarks en el VCS (Git).
- No utilizó la última versión LTS de Python como versión mínima, lo que podría llevar a inconsistencias entre desarrolladores que tengan instaladas distintas versiones de este lenguaje.
- No documentó los tests.

- Separó correctamente las distintas capas de la aplicación, aunque una estructura Feature-Sliced Design habría sido más conveniente para la DX.
- Utilizó comparaciones con literales en vez de recurrir a constantes.
- Acopló los textos al código de la UI, dificultando que en el futuro se agreguen múltiples idiomas al juego.
- Utilizó Type Hints.
- No utilizó Named Parameters, lo que dificulta la implementación de los métodos que tienen tres o más parámetros.
- Inhabilitó mensajes del linter para evitar alertas en ciertas líneas.
- Si un Worker falla, los workers que iniciaron previamente no se detienen, quedando a la espera de que se le asignen tareas.

Conclusiones

Reflexiones sobre ventajas y limitaciones de usar IA en programación concurrente

Nuestra reflexión principal es que la IA acelera mucho la construcción de un sistema concurrente, pero que el programa funcione y pase las pruebas no alcanza para demostrar que respeta sus requisitos. Los hallazgos de este proyecto desarrollado con Astra ilustran bien esa diferencia.

La IA aporta ventajas claras:

- **Reduce el trabajo:** ayuda a implementar workers, canales de comunicación, sincronización y cierre de recursos.
- **Facilita explorar diseños:** permite discutir qué tareas conviene ejecutar en paralelo y cómo coordinar sus resultados.
- **Sirve como apoyo para code reviews:** puede proponer escenarios de bloqueo, interrupciones y condiciones de carrera que suelen quedar fuera del recorrido normal.

Su limitación más importante es que puede producir una solución convincente que dependa de supuestos incorrectos.

No atribuiría estos errores exclusivamente a este modelo (Astra), porque también son errores humanos que suelen ocurrir. En programación concurrente, el aporte humano sigue siendo decisivo para definir qué debe cumplirse, bajo qué condiciones y con qué evidencia se puede afirmar que se cumplió.

Referencias

No aplica.